

# EventVibe — Technical Project & Viva Report

Full-Stack Event Discovery & Real-Time Tracking Platform | Quantiphi Assessment

## 1. Executive Summary

EventVibe is a complete, production-ready full-stack event discovery and tracking platform built using React, Vite, FastAPI, and MongoDB. The system enables users to discover events, manage RSVPs, track personal dashboards, invite friends via trackable links, view real-time WebSocket attendance updates, and query events using an intelligent deterministic chat assistant.

## 2. What We Built (Features & Capabilities)

- **Event Discovery & API Feed Integration:** Integrates with the Ticketmaster Discovery API on the backend with an automated fallback to structured mock events across Music, Technology, Sports, Arts, and Food across major cities (Mumbai, Pune, New York, Bangalore).
- **Custom React Event Calendar:** A zero-dependency interactive monthly calendar highlighting event dates and supporting date-filtered event views.
- **Server-Authoritative RSVP & Dashboard:** Single-click RSVP confirmation, duplicate RSVP rejection (409 Conflict), and an RSVP dashboard computing total and upcoming events.
- **Friend Invitation System:** Generates unique shareable links (/invite/:token) allowing RSVP'd users to invite friends.
- **Click Tracking vs. Friends Attending:** Deduplicates visitor clicks (clickCount) separately from confirmed friend RSVPs (friendsAttending). Clicks never inflate attendance.
- **Real-Time WebSockets:** Live bidirectional WebSocket connections on /ws broadcast attendance changes to instantly update event cards across clients without full page reloads.
- **User Profile & Reminders:** User preferences for event reminders with customizable lead times (validated from 1 to 10,080 minutes).
- **Intelligent Chat Assistant:** Regex/keyword-driven chat assistant parsing natural queries for category, city, date, and user RSVP records.

## 3. Technical Architecture & Stack

The application utilizes a clean Client-Server architecture where all business logic, validation, and attendance calculations reside strictly on the server.

| Layer    | Technologies Used                                 | Key Responsibilities                                                                              |
|----------|---------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Frontend | React 18, Vite, React Router v6, Axios, Plain CSS | Renders UI, collects user input, manages WebSocket connection, routes pages.                      |
| Backend  | Python 3.14, FastAPI, Uvicorn, Pydantic v2, HTTPX | Validates inputs, enforces database constraints, fetches Ticketmaster API, handles WS broadcasts. |
| Database | MongoDB, PyMongo, mongomock fallback              | Stores collections for users, rsvps, invites, and invite_clicks with unique compound indexes.     |

## 4. Key Engineering & Design Decisions

- **Atomic Unique-Index Deduplication:** Instead of fragile check-then-insert application logic, uniqueness for RSVPs (userId, eventId), invites (token), and clicks (token, visitorId) is enforced at the database level via MongoDB unique compound indexes catching DuplicateKeyError.
- **Immutable Event Snapshots:** To insulate user RSVPs and invitations against external API changes or expired Ticketmaster events, complete event metadata (title, venue, city, date, image) is snapshotted upon RSVP creation.
- **Ticketmaster API Backend Isolation:** The Ticketmaster API key is stored exclusively in server environment variables and never exposed to the client. Normalization into a unified structure occurs entirely inside backend services.
- **Server-Side Business Authority:** Clients never calculate attendance, friend counts, or upcoming statistics. The server performs all calculations and returns authoritative read-only numbers.

## 5. Verification & Testing

The project was verified through a comprehensive automated smoke test suite (backend/smoke\_test.py) covering 14 critical end-to-end assertions:

- [PASSED] 1. /api/health responds with HTTP 200 OK.
- [PASSED] 2 & 3. /api/events returns normalized feed (Ticketmaster / Mock fallback).
- [PASSED] 4. Demo user can successfully RSVP to an event.
- [PASSED] 5. Duplicate RSVP correctly rejected with HTTP 409 Conflict.
- [PASSED] 6. Dashboard returns accurate total and upcoming confirmed RSVPs.
- [PASSED] 7. User can generate a unique invite link.
- [PASSED] 8. Invite GET endpoint returns valid inviter and event details.
- [PASSED] 9. Visitor click tracking deduplicates repeat clicks from the same visitor ID.
- [PASSED] 10 & 11. Friend RSVP via invite link successfully increments friendsAttending by 1.
- [PASSED] 12. Duplicate friend RSVP rejected with 409 without inflating attendance.
- [PASSED] 13. RSVP cancellation decrements friendsAttending back to 0.
- [PASSED] 14. Invalid invite tokens return HTTP 404 Not Found.

**Frontend Build:** Ran npm run build (vite build) — 100 modules transformed, 0 errors.

## 6. Viva Preparation Guide

### 10-Line Architecture Summary for Viva:

1. React + Vite SPA renders UI components and handles client routing.
2. Axios sends REST HTTP calls to FastAPI server endpoints on Uvicorn.
3. FastAPI enforces input validation using Pydantic schemas.
4. Ticketmaster Discovery API queried on backend via HTTPX (8s timeout).
5. Automatic fallback to mock events if API key is missing or request fails.
6. MongoDB stores users, rsvps, invites, and invite\_clicks collections.
7. MongoDB unique compound indexes prevent duplicate RSVPs and clicks.
8. Real-time updates broadcast over WebSockets (/ws) on RSVP actions.
9. React WebSocket client listens and updates affected cards live.
10. Deterministic chat assistant parses natural queries via regex matching.

## Core Viva Questions & Concise Answers:

### Q1: Why React + Vite for the frontend?

React provides declarative component rendering perfect for updating cards and calendar states. Vite provides instant HMR and optimized production builds.

### Q2: Why FastAPI for backend?

FastAPI offers high speed, automatic Pydantic validation, native WebSocket support, and auto-generated Swagger documentation.

### Q3: Why MongoDB?

MongoDB's flexible document structure naturally fits nested event snapshots and user reminder settings without requiring complex relational joins.

### Q4: Why keep Ticketmaster API on backend?

Keeps API keys secret, prevents CORS issues, normalizes raw responses, and handles fallback to mock data transparently.

### Q5: How are duplicate RSVPs prevented?

By creating a unique compound index on (userId, eventId) in MongoDB. Attempting a duplicate raises DuplicateKeyError, returning HTTP 409 Conflict.

### Q6: Difference between clickCount and friendsAttending?

clickCount tracks unique link visitors via (token, visitorId) compound index. friendsAttending counts actual confirmed RSVPs created through the invite.

### Q7: How do real-time updates work?

When an RSVP occurs, websocket\_manager.py broadcasts an rsvp\_updated JSON payload to all /ws clients. React updates the affected card state dynamically.

### Q8: How does the chat assistant work?

chat\_service.py uses regex pattern matching to parse categories, cities, dates, and personal user RSVP records against MongoDB.

### Q9: Why store eventSnapshots in RSVPs?

External API data can change or expire. Storing an immutable snapshot guarantees the user's dashboard displays consistent data.

### Q10: Why is PyMongo synchronous in FastAPI?

PyMongo is simple and reliable for MVPs, executing in <1ms for local/in-memory database operations.